Object-Oriented Programming (OOP)

OOP from the Ground Up

cs107e

OO Programming Languages

C++

Python

Javascript

...

Why OOP?

Abstraction - Abstract data types

- **Different implementations with the same interface**

Encapsulation

- **Expose and hide internal variables and functions**

Inheritance

- **Extend an existing implementation**

Polymorphism

- **Generic functions that take more than one type**

Inventors

Kristen Nygaard **Ole-Johan Dahl**

Their Why

Simula: A language that could serve the dual purpose of system modeling and simulation programming

Collection of objects match our mental model of the system, like we use categories for objects, parts and sub-parts to describe the world

Capture state and behavior

Monsters University (2013)

pbrt

Table 1.1: Main Interface Types. Most of pbrt is implemented in terms of 10 key abstract base classes, listed here. Implementations of each of these can easily be added to the system to extend its functionality.

Base class	Directory	Section
Shape	shapes/	3.1
Aggregate	accelerators/	4.2
Camera	cameras/	6.1
Sampler	samplers/	7.2
Filter	filters/	7.8
Material	materials/	9.2
Texture	textures/	10.3
Medium	media/	11.3
Light	lights/	12.2
Integrator	integrators/	1.3.3

```
class Shape …

class Sphere : class Shape
class Cylinder : class Shape
class Cone : class Shape

class Triangle : class Shape
class Mesh : class Shape

class Fractal : class Shape

…
```

System on a Chip (SOC)

Core

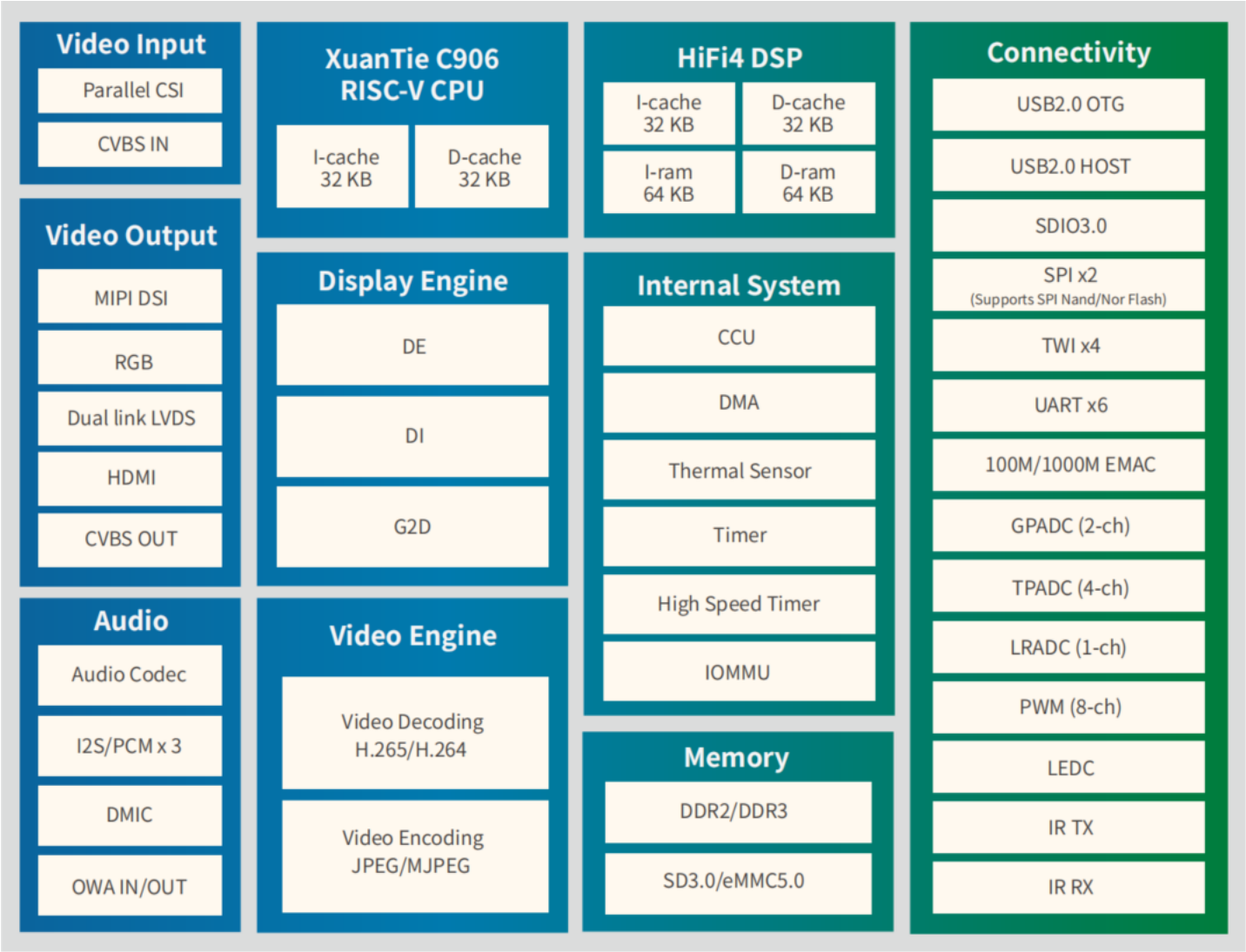
- **RISCV**
  - **RV32I**
  - **RV64E**

Memory

- **EEPROM**
- **RAM**

Peripherals

- **GPIO**
- **UART**



Mango Pi - Allwinner D1

```
// GPIO module

void gpio_init(void);

void gpio_set_function(gpio_id_t pin,
    unsigned int function);
unsigned int gpio_get_function(gpio_id_t pin);

void gpio_set_input(gpio_id_t pin);
void gpio_set_output(gpio_id_t pin);

int gpio_read(gpio_id_t pin);
void gpio_write(gpio_id_t pin,
    int val);
```

```
// UART module

void uart_init(void);

int uart_getchar(void);

int uart_putchar(int ch);
void uart_flush(void);
```

```
class Peripheral {
 void init(void);
 …
}

class GPIO : class Peripheral

class UART : class Peripheral
```

```
// ps2 - why ps2_new?

ps2_device_t *ps2_new(
    gpio_id_t clock,
    gpio_id_t data);

unsigned char ps2_read(
    ps2_device_t *dev);

bool ps2_write(
    ps2_device_t *dev,
    unsigned char command);
```

```c
struct ringbuffer {
    int entries[LENGTH];
    int head, tail;
};
typedef struct ringbuffer rb_t;

rb_t *rb_new(void) {
    // allocate storage to create an instance of struct
    rb_t *rb = malloc(sizeof(struct ringbuffer));
    assert(rb != NULL);
    // initialize fields
    rb->head = rb->tail = 0;
    // return instance
    return rb;
}

bool rb_empty(rb_t *rb);
bool rb_full(rb_t *rb);
bool rb_enqueue(rb_t *rb, int elem);
bool rb_dequeue(rb_t *rb, int *p_elem);
```

```
// V2
struct ringbuffer {
    int entries[LENGTH];
    int head, tail;

    bool (*rb_empty)(rb_t *rb);
    bool (*rb_full)(rb_t *rb);
    bool (*rb_enqueue)(rb_t *rb,
        int elem);
    bool (*rb_dequeue)(rb_t *rb,
        int *p_elem);
};

rb = rb_new();
rb->enqueue(rb, 'a');
```

```
// V3
struct class_ringbuffer {
    bool (*rb_empty)(rb_t *rb);
    bool (*rb_full)(rb_t *rb);
    bool (*rb_enqueue)(rb_t *rb, int elem);
    bool (*rb_dequeue)(rb_t *rb, int *p_elem);
}
struct class_ringbuffer rb_table = {
    rb_empty;
    rb_full;
    rb_enqueue;
    rb_dequeue;
}
struct ringbuffer {
    int entries[LENGTH];
    int head, tail;
    void *fn_table; // = &rb_table
};
```

```
class RingBuffer {
    int entries[LENGTH];
    int head, tail;

    bool rb_empty() { … };
    bool rb_full() { … };
    bool rb_enqueue(int elem) { … }
    bool rb_dequeue(int *p_elem) { … };
};

rb = new RingBuffer();
rb->enqueue(1);
```

Classes in C (1982)

Bjarne Stroustrup

Design Principles

Must work with existing infrastructure

- `cfront` **- translate from C++ to C**
- **Use existing tools such as** `ld, gdb`

Method invocation should be as efficient as a function call

```
class A {
  public:
    int x;
    int y;
    int f(int);
};

int A::f(int c) {
    return x+y+c;
}
```

```
// A.list
//  Same cost as a function call!
//. this is passed in a0

0000000000000000 <_ZN1A1fEi>:
   0:  ff010113          add  sp,sp,-16
   4:  00813423          sd s0,8(sp)
   8:  01010413          add  s0,sp,16

   c:  00052783          lw a5,0(a0)
  10:  00452703          lw a4,4(a0)
  14:  00e787bb          addw a5,a5,a4
  18:  00b7853b          addw a0,a5,a1

  1c:  00813403          ld s0,8(sp)
  20:  01010113          add  sp,sp,16
  24:  00008067          ret
```

C++ Name Mangling

What is `_ZN1A1fEi` !!?

- `_ZN` - **compiler generated name**
- `1 A 1 f E > A_f`
- `i - int`

Why?

- **Classes**
- **Function overloading**
- **Namespaces**

```
// Inheritance
class A {
  public:
    int x;
    int y;
    int f(int);
};

class B : public A {
  public:
    int z;
    int f(int);
};

int A::f(int c) {
    return x+y+c;
}

int B::f(int c) {
    return 2*x+y+c;
}
```

```
// Abstract base class
class Base {
  public:
    int x;
    int y;
    virtual int f(int) = 0;
    // = 0 forces derived classes to implement f
    // Base cannot be instantiated itself
};

class A : public Base {
  public:
    int f(int);
};

class B : public Base {
  public:
    int f(int);
};
```

```
int A::f(int c) {
    return 2*x+y+c;
}

int B::f(int c) {
    return x+2*y+c;
}
```

```
// Polymorphism

// base can be any derived class
int call_f(Base *base, int c) {
    return base->f(c);
}

int main() {
    A a;
    int x = call_f(&a, 1);
    B b;
    int y = call_f(&b, 1);
}

// How does the compiler know
// to call A.f vs B.f?
```

```
0000000000000000 <_Z6call_fP4Basei>:
   0:   ff010113            add sp,sp,-16
   4:   00113423            sd  ra,8(sp)
   8:   00813023            sd  s0,0(sp)
   c:   01010413            add s0,sp,16
; call virtual function (base in a0)
  10:   00053783            ld  a5,0(a0)
  14:   0007b783            ld  a5,0(a5)
  18:   000780e7            jalr a5

; determine which f to call at run-time
; first entry in class base
;   points to a table of function pointers
; first entry in table of functions
;   points to f
```

// Virtual function tables

```
class Base
  0 *vtable
  8 int x
 12 int y
```

```
Base vtable
0 (*f)(int) =_ZN4Base1fEi
```

```
class A
  0 *vtable
  8 int x
 12 int y
```

```
A vtable
0 (*f)(int)=_ZN1A1fEi
```

```
class B
  0 *vtable
  8 int x
 12 int y
```

```
B vtable
0 (*f)(int)=_ZN1B1fEi
```

Pure OO

Everything is an object!

Dynamic typing examples

- **Javascript**
- **Python**

Javascript

```
// dictionaries

d = { x: 1, y : 2 };

d.x;
d[“x”];

“x” in d;

for (key in d) …

e = { x: 1, y : 2, f() { return x+y; }; };

e.f();
```

```
// dictionaries -> objects

o = {
  x: 1,
  y: 2,
  __proto__: {
    y: 3,
    z: 4,
  },
};

// Search up prototype chain
```

```
// A constructor function
function A(x, y) {
  this.x = x;
  this.y = y;
}

A.prototype.f = function () {
  return this.x + this.y;
};

a = new A(1,2);

// create object and pass to A as this
// this.__proto__ = A.prototype
// return this
```

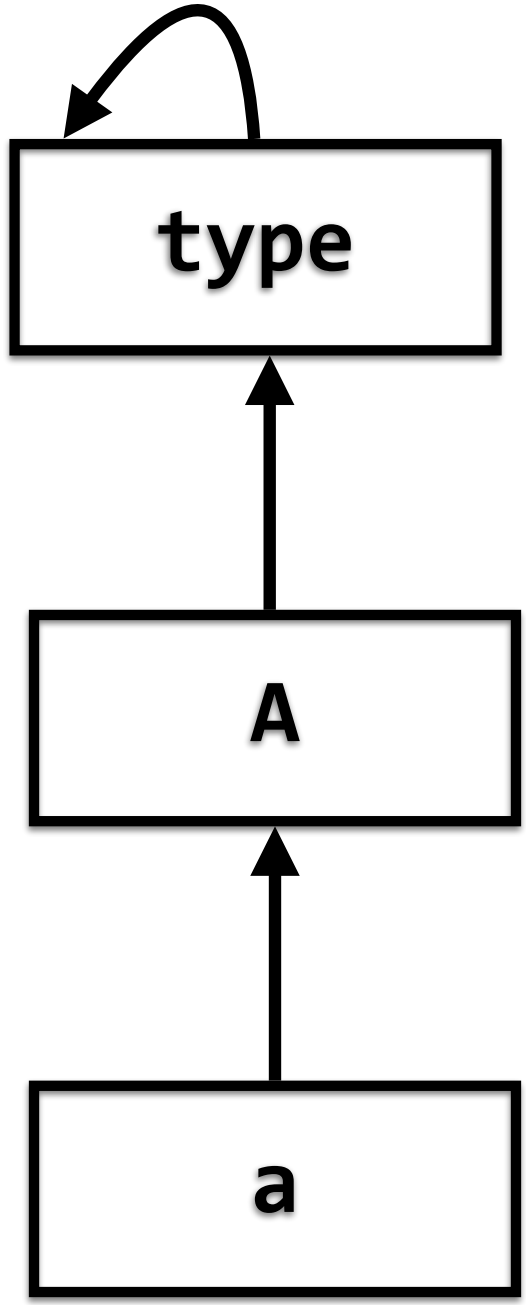
You make prototype objects, and then ... make new instances. Objects are mutable in JavaScript, so we can augment the new instances, giving them new fields and methods. These can then act as prototypes for even newer objects. We don't need classes to make lots of similar objects... Objects inherit from objects. What could be more object oriented than that?[

Douglas Crockford

Python

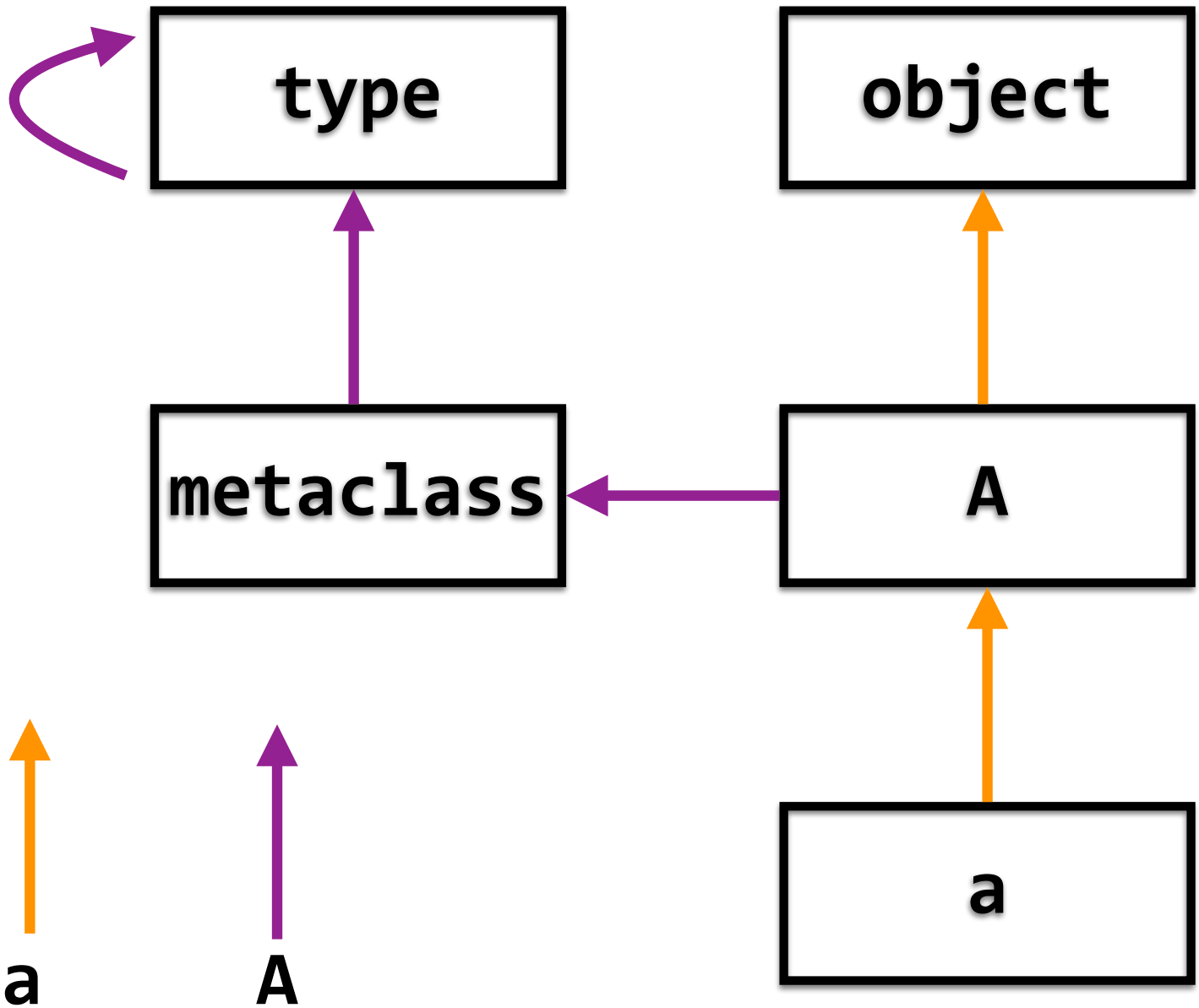
```
class A(object):
  def __init__(self, x, y):
    self.x = x
    self.y = y

a = A(1,2)

type(a)    # <class ‘__main__.A'>
type(A)    # <class ‘type'>
type(type) # <class ‘type’>

isinstance(a, A)    # True
isinstance(A, type) # True
issubclass(A, object) # True
```

Class Hierarchy



type is a metaclass
(type of a class)

Inheritance Hierarchy



Summary

Course goals

- **Understand how computers work**
- **Master your tools**
  - **Understand your tools!**

Build more powerful systems and tools on top of other tools

Raise the abstraction